

02DataSplit

February 6, 2026

```
[1]: # import packages
import pandas as pd
import numpy as np

# read in unrefined dataset
tourney_games = pd.read_csv("tourneyGames.csv")

[2]: # drop unnecessary columns
cols_to_drop = list(range(12,13)) + list(range(101, 157)) + list(range(159, 172)) + list(range(260, 316)) + list(range(318, 330))
tourney_games = tourney_games.drop(tourney_games.columns[cols_to_drop], axis=1)

[3]: # create target (1 if better seed won, 0 if worse seed won)
def lower_seed_won(row):
    if row["winnerTeamId"] == row["team1Id"]:
        winning_seed = row["team1_Seed"]
        losing_seed = row["team2_Seed"]
    else:
        winning_seed = row["team2_Seed"]
        losing_seed = row["team1_Seed"]

    return int(winning_seed < losing_seed)

tourney_games["y"] = tourney_games.apply(lower_seed_won, axis=1)

# identify team stat columns
team1_cols = [c for c in tourney_games.columns if c.startswith("team1_")]
team2_cols = [c for c in tourney_games.columns if c.startswith("team2_")]

# find shared stats between team1 and team2
stats = sorted(
    set(c.replace("team1_", "") for c in team1_cols)
    & set(c.replace("team2_", "") for c in team2_cols)
)

# create difference stats (team1 - team2)
for stat in stats:
```

```

    tourney_games[f"{stat}_diff"] = tourney_games[f"team1_{stat}"] -
↳ tourney_games[f"team2_{stat}"]

# keep identifier data
base_cols = [
    "Season", "DayNum",
    "WTeamID", "LTeamID",
    "WScore", "LScore",
    "team1Id", "team2Id",
    "y", "winnerTeamId",
    "team1Name", "team2Name",
    "team1_Seed", "team2_Seed"
]

# keep difference columns
diff_cols = [f"{stat}_diff" for stat in stats]

# final dataset creation
final_df = tourney_games[base_cols + diff_cols]

final_df

```

```

[3]:      Season  DayNum  WTeamID  LTeamID  WScore  LScore  team1Id  team2Id  y  \
0      2003      134      1421      1411        92        84      1411      1421  0
1      2003      136      1112      1436        80        51      1112      1436  1
2      2003      136      1113      1272        84        71      1113      1272  0
3      2003      136      1141      1166        79        73      1141      1166  0
4      2003      136      1143      1301        76        74      1143      1301  1
...      ...      ...      ...      ...      ...      ...      ...      ...
1348    2024      146      1301      1181        76        64      1181      1301  0
1349    2024      146      1345      1397        72        66      1345      1397  1
1350    2024      152      1163      1104        86        72      1104      1163  1
1351    2024      152      1345      1301        63        50      1301      1345  1
1352    2024      154      1163      1345        75        60      1163      1345  0

```

```

      winnerTeamId  ... Raw Defensive Efficiency Rank_diff  \
0      1421  ... -165
1      1112  ... -34
2      1113  ... 123
3      1141  ... 159
4      1143  ... -39
...      ...      ...
1348      1301  ... -106
1349      1345  ... 45
1350      1163  ... 261
1351      1345  ... 85
1352      1163  ... -43

```

	Raw Offensive Efficiency_diff	Raw Offensive Efficiency Rank_diff	\
0	1.6	-43	
1	9.0	-133	
2	3.2	-42	
3	-5.4	34	
4	-1.7	19	
...	...	...	
1348	8.4	-90	
1349	7.1	-40	
1350	-1.6	1	
1351	-10.3	97	
1352	3.7	-3	

	Raw Tempo_diff	Raw Tempo Rank_diff	Seed_diff	StlRate_diff	\
0	1.1	-36	0	-0.009500	
1	10.0	-263	-15	0.006100	
2	-0.7	25	3	-0.026100	
3	3.3	-100	5	-0.020000	
4	2.2	-96	-1	-0.020600	
...	...	...	...	...	
1348	-1.4	95	-7	-0.007230	
1349	-1.9	114	-1	-0.031880	
1350	7.9	-296	3	-0.000093	
1351	0.2	-22	10	0.025653	
1352	-2.3	115	0	0.014265	

	TOPct_diff	Tempo_diff	eFGPct_diff
0	-1.005500	1.1023	1.696500
1	-2.422900	9.9893	2.742900
2	0.126500	-0.6498	1.957200
3	5.750300	3.3585	-0.099900
4	-1.314400	2.1439	-1.044500
...	...	...	...
1348	0.653412	-1.4484	3.893735
1349	1.966710	-1.9514	4.685253
1350	1.418475	7.8314	-0.594558
1351	-2.647674	0.2654	-5.084930
1352	-1.943160	-2.2433	1.097619

[1353 rows x 104 columns]

```
[4]: # export dataset to csv
final_df.to_csv("matchupDiff.csv")
```

```
[5]: # split into training and testing
train_df = final_df[(final_df["Season"] >= 2003) & (final_df["Season"] <= 2019)]
```

```
test_df = final_df[(final_df["Season"] >= 2021) & (final_df["Season"] <= 2024)]

# separate features and target
X_train = train_df.drop(columns=["y"])
y_train = train_df["y"]

X_test = test_df.drop(columns=["y"])
y_test = test_df["y"]
```

[]: